

**Document number:** P1651R0  
**Date:** 2019-06-07  
**Audience:** Library Evolution Working Group  
**Reply-to:** Tomasz Kamiński <tomaszka@gmail.com>

# bind\_front should not unwrap reference\_wrapper

## 1. Introduction

This paper proposes a change in the behaviour of the `std::bind_front` in regards to the bound arguments of the `std::reference_wrapper<T>` type — the arguments should not be unwrapped (passing `rw.get()` to underlying callable), and should be propagated unchanged instead (passing `rw` unmodified to underlying callable).

This change is the result of analysis of use case provided by the Abseil team, that I have not considered during the initial design. The main motivation of the change is to reduce the extent of damage that may be implied by code that assumes "incorrect" behavior.

| Before: <code>bind_front</code> has unwrapping semantic                                                                                                                                                                                  | After: <code>bind_front</code> has propagation semantic                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Code that assumes propagation semantic may: <ul style="list-style-type: none"><li>• not compile (static error),</li><li>• create silent compiles (performance impact),</li><li>• use dangling references (undefined behavior).</li></ul> | Code that assumes unwrapping semantic may: <ul style="list-style-type: none"><li>• not compile (static error).</li></ul> |
| Fixing code requires the implementation of a custom binder.                                                                                                                                                                              | Fixing code requires a cast to desired view type.                                                                        |

Changes proposed in this paper need to be considered in the C++20 timeline, as they would constitute breaking change after the publication of standard in the current form.

## 2. Revision history

### 2.1. Revision 0

Initial revision.

## 3. Motivation and Scope

This paper discusses the behavior of the code that uses `std::bind_front` in conjunction with `std::reference_wrapper` instantiations. To illustrate let's consider the following code:

```
Thingy thingy;  
auto boundFunctor = std::bind_front(func, std::ref(thingy));
```

There are two possible and conflicting behaviors of the `boundFunctor()` invocation (for purpose of presentation we refer to `bound reference_wrapper` as `thingyRef`):

- *unwrapping*: `func(thingyRef.get())`, i.e. passing `Thingy &` — mimics behaviour of `std::bind_front` w.r.t. `reference_wrapper`,
- *propagation*: `func(thingyRef)`, i.e. passing `std::reference_wrapper<Thingy>` — `std::reference_wrapper` is treated as any other class.

The difference between above behavior is subtle and manifest itself only in specific corner case — it's usually not observable in the user code, due to the implicit conversion from `std::reference_wrapper<Thingy>` to `Thingy&`.

### 3.1. Unwrapping use case: Double conversion

One of the situations when the difference between unwrapping and propagation semantic is visible, is when the functor is bound with the reference to the object (e.g. `std::reference_wrapper<std::string>`), and the code accepts an view to it (e.g. `std::string_view`). To illustrate let's consider:

```

void functionAcceptingStringView(std::string_view);
void functionAcceptingSpanOfIntegers(std::span<int>);

std::string s;
auto fs = std::bind_front(&functionAcceptingStringView, std::ref(s));
std::vector<int> v;
auto fv = std::bind_front(&functionAcceptingSpanOfIntegers, std::ref(v));

```

With the unwrapping (current) semantic, both `fs()` and `fv()` compiles correctly, as `std::string_view` can be implicitly constructed from `std::string&` and `std::span<int>` can be implicitly constructed from `std::vector<int>&`. In case if propagation semantic was provided, both invocation would not compile, as they would require two user defined conversion to be performed:

- `std::reference_wrapper<std::string> → std::string& → std::string_view` for `fs()`, and
- `std::reference_wrapper<std::vector<int>> → std::vector<int>& → std::span<int>` for `fv()`.

However, both of above compilation issues can be easily fixed by binding the desired view type instead of using `std::ref`:

```

auto fs = std::bind_front(&functionAcceptingStringView, std::string_view(s));
auto fv = std::bind_front(&functionAcceptingSpanOfIntegers, std::span(v));

```

### 3.2. Propagation use case: Function currying

The difference between the propagation is unwrapping semantic, is that the former support rebinding of the argument. This can be best illustrated with the implementation of the functor `PartialApply` that implements function currying in C++:

```

template <typename F>
struct PartialApply {
    PartialApply(F f) : f(f) {}
    F f;

    template <typename... A> auto operator()(A const&... a) const {
        if constexpr (std::is_invocable<F const&, A const&...>::value) {
            return f(a...);
        } else {
            return bind_front(*this, a...);
        }
    }
};

```

The intent of the above code is that the expression `PartialApply(func)(a)(b)` is either:

- calling `func(a, b)` if that is well-formed,
- returning callback that binds `func` with `a` and `b` and accepts consecutive arguments (*rebouncing*).

The above implementation works flawlessly in case of the propagation semantic, however, it fails in the case of unwrapping is used — in the case when rebinding is performed, copy of arguments that were originally passed via `std::ref` is made.

In the most optimistic scenario the above issue will manifest as compilation error. This happens when move only type is passed by reference:

```

std::unique_ptr<Thingy> thingy;
auto func = [](std::unique_ptr<Thingy>&, int) {};

PartialApply(func)(std::ref(thingy))(10);

```

In the case of copyable types, silently copy of the object will be created. This, of course, may have negative performance implication, but in the worst case can lead to dangling references:

```

std::string str;
auto func = [](std::string& s, int) -> std::string& { return s; };

std::string& sref = PartialApply(func)(std::ref(s))(10); // sref refers to copy of str stored in PartialApply
sref.push_back('a'); // use of dangling reference

```

Finally, the author is not aware of the way of fixing `PartialApply` than reimplementing custom `bind_front` alternative that has propagation semantic.

### 3.3. Historical design note on [LWG 2219](#)

The prominent use case for partial function application functions (like `std::bind_front` and `std::bind`), is to provide a helper to compose given method on the class, with a specific object. In case if invocation should be performed on existing object instance (not a

copy), a `std::reference_wrapper` was used as follows:

```
auto bound = std::bind_front(&Object::method, std::ref(instance));
```

In the time when the `std::bind_front` was originally proposed (also applies to `std::bind`), the above code to work properly required unwrapping semantic (at least the for first argument). This was changed with the resolution of [LWG 2219: INVOKE-ing a pointer to member with a reference wrapper as the object expression](#), that have introduced dedicated handling for `std::reference_wrapper` in *INVOKE*, and unwrapping of `std::reference_wrapper` is no longer required to achieve this functionality.

## 4. Design Decisions

### 4.1. Preference for compilation errors

As indicated in the motivation section, the difference between unwrapping and propagation semantic manifest itself in a very specific scenario. As a consequence, the user, unaware of this specific behavior, may accidentally create an erroneous code that depends on semantic other than one supplied by the standard. In that case, it would be desired that error is detected as early as possible, preferably at compile time.

In light of the above, this paper proposes to switch to propagation semantic, as code that assumes unwrapping becomes ill-formed. With the current behavior (unwrapping), the code that assumes propagation may lead to runtime bugs.

### 4.2. Preserving other behavior

The change proposed in this paper has no impact on user-visible behaviour of `std::bind_front`, despite the fact that functors returned by this functor will now store `std::reference_wrapper<T>` instead of `T&`, as:

- functors produced by `std::bind_front` are not assignable, so the difference in assignment semantic is not visible to the user,
- `reference_wrapper`, like references, is required to be trivially copyable,
- `bind_front` is not currently usable in `constexpr` context.

In addition the [P1065: constexpr INVOKE](#) applies `constexpr` to `std::reference_wrapper`.

## 5. Proposed Wording

The proposed wording changes refer to N4810 (C++ Working Draft, 2019-03-15).

Apply following changes to section [func.bind\_front] Function template `bind_front`:

```
template <class F, class... Args>
    unspecified bind_front(F&& f, Args&&... args);
```

In the text that follows:

- `g` is a value of the result of a `bind_front` invocation,
- `FD` is the type `decay_t<F>`,
- `fd` is a target object of `g` ([func.def]) of type `FD` initialized with *initializer* (`std::forward<F>(f)`),
- `BoundArgs` is a pack that denotes ~~`std::unwrap_ref_decay_t<Args>...`~~,
- `bound_args` is a pack of bound argument entities of `g` ([func.def]) of types `BoundArgs...` initialized with *initializers* (`std::forward<Args>(args)`)... respectively, and
- `call_args` is an argument pack used in a function call expression ([expr.call]) of `g`,

Update the value of the `__cpp_lib_bind_front` in table "Standard library feature-test macros" of [support.limits.general] to reflect the date of approval of this proposal.

## 6. Implementability

This paper can be implemented by simply replacing `unwrap_ref_decay_t` with `decay_t` in example implementation from "Implementability" section of the [P0365R5: Simplified partial function application](#) paper.

## 7. Acknowledgements

Titus Winters and Abseil team for providing the feedback on the `bind_front` and code example that motivated creation of this paper.

Samuel Benzaquen for providing feedback and corrections for this paper.

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal and author's participation in standardization committee.

## 8. References

1. Jonathan Wakely, Issue 2219. *INVOKE*-ing a pointer to member with a `reference_wrapper` as the object expression, (LWG2219, <https://wg21.link/lwg2219>)
2. Barry Revzin, `constexpr INVOKE`, (P1065, <https://wg21.link/p1065>)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4810, <https://wg21.link/n4810>)
4. Tomasz Kamiński, Simplified partial function application, (P0356R5, <https://wg21.link/p0356r5>)